

KEIKEU / FRONTEND TOOLCHAIN / BEGINNER

给新人的 npm 手册

只学 keikeu 今天真正使用的 Node、npm 与前端命令：先核对版本，按锁文件安装，再分清 Vite 页面和完整桌面应用。

KEIKEU CONTRIBUTORS 2026-08-11 NODE 22.23.2 NPM 10.9.8 PDF 版

先记住三件事

在 keikeu 里，npm 只管理 frontend/ 的 JavaScript 依赖与脚本。它不是 Python 后端，也不会替你完成 Rust、Sidecar 或产品验收。

- 01 从仓库根目录运行命令时，始终带上 --prefix frontend。
- 02 package.json 写直接需求，lockfile 锁完整依赖树，node_modules 可以重建。
- 03 网页能打开只证明 Vite 在工作；完整桌面链路还需要 Tauri、Rust 和 Python Sidecar。

CONTENTS

阅读路线

- | | | | |
|----|-------------|----|---------|
| 01 | npm 在项目里的位置 | 02 | 五个文件与目录 |
| 03 | 第一次安装 | 04 | 日常命令 |
| 05 | 走到完整桌面应用 | 06 | 依赖变更纪律 |
| 07 | 故障与证据边界 | | |

npm 在项目里的位置

Node 运行前端工具，npm 安装这些工具并调用项目脚本；keikeu 的业务核心仍在 Python，桌面宿主仍在 Rust/Tauri。

先把四层分开

01	Vue 描述界面组件、状态与交互。
02	Vite / Vitest 开发、构建和测试 Vue 代码。
03	Tauri / Rust 提供桌面窗口、生命周期与本机能力。
04	Python Sidecar 执行 Vault、Markdown、索引与业务规则。

npm 主要站在第二层，也负责调用本地安装的 Tauri CLI。它不会安装 Python 包，不会运行 Cargo 测试，也不是一个常驻 Web 后端。

当前直接依赖

用途	当前包	新人怎么理解
界面运行时	<code>vue 3.5.40</code>	组件与响应式状态。
Tauri 前端接口	<code>@tauri-apps/api 2.11.1</code> <code>@tauri-apps/plugin-dialog 2.7.2</code>	让 WebView 中的界面调用被允许的桌面能力。
开发与构建	<code>vite 8.1.5</code> <code>@vitejs/plugin-vue 6.0.8</code>	开发服务器与静态资源构建。
测试	<code>vitest 4.1.10</code> <code>@vue/test-utils 2.4.11</code> <code>jsdom 29.1.1</code>	在模拟浏览器环境中检查 Vue 行为。

用途	当前包	新人怎么理解
桌面命令	@tauri-apps/cli 2.11.4	把 npm 命令交给 Tauri 与 Cargo。

版本不是 ROAD 名称

npm 项目当前名为 `keikeu-desktop`，manifest 版本是 `0.1.0`。这个字段不能直接证明 Road v0.6、发布候选或用户可见版本已经成立。

CHAPTER 02

五个文件与目录

只要能区分“需求、冻结结果、安装结果、构建结果”，就不会把依赖问题修成更大的依赖问题。

frontend/package.json

直接依赖、精确工具版本和脚本入口。`private: true` 防止误发布，`type: module` 表示前端 JavaScript 使用 ESM。

frontend/package-lock.json

npm 解析后的完整依赖树，当前格式为 lockfile v3。它必须进入 Git，团队与 CI 才能重建同一棵树。

frontend/node_modules/

本机安装结果，体积大、可重建、已被 Git 忽略。不要手改，也不要提交。

frontend/dist/

Vite 的构建输出，运行 `npm run build` 后生成，同样不进入 Git。

frontend/vite.config.js

当前开发服务器固定在 `127.0.0.1:1420`，并启用 `strictPort`；Vitest 使用 `jsdom` 环境。

一句话地图

```
package.json + package-lock.json
|
└─ npm ci → node_modules/

Vue source — npm run build → dist/
```

谁该进 GIT

提交 `package.json` 与 `package-lock.json`；不提交 `node_modules/` 或 `dist/`。删除 lockfile 不是“清缓存”，而是放弃当前冻结结果。

CHAPTER 03

第一次安装

先核对真正运行的版本，再用冻结安装；不要假设版本文件或包管理器会自动替你守好这道门。

第一步：从仓库根目录核对版本

```
node --version
npm --version
```

当前前端契约应输出：

```
v22.23.2
10.9.8
```

`frontend/package.json` 的 `engines` 同样写着这两个精确值，但 `engines` 不一定是硬失败闸门；命令输出才是你此刻真正使用的工具。

已知漂移 · 不要藏起来

当前 `frontend/package.json`、lockfile、README、PROJECT 与 Road v0.6 都写 Node `22.23.2`，但仓库根目录 `.node-version` 仍是旧值 `22.23.1`。新人以当前 manifest 与 Road 契约为准；校准该文件应当是独立、可审阅的任务，不在安装时偷偷改。

第二步：按 lockfile 安装

```
npm --prefix frontend ci
```

`npm ci` 会：

- 要求 `package.json` 与 lockfile 一致；
- 先删除现有 `frontend/node_modules/`，再重建；
- 按 lockfile 安装，不更新两个声明文件；
- 首次安装或缓存未命中时访问 npm registry。

因此它适合新人、CI 和“把本机恢复到仓库声明状态”。不要给它加 `sudo`，也不要为了消除报错先删 lockfile。

第三步：只读核对顶层依赖

```
npm --prefix frontend ls --depth=0
```

成功时应看到本手册第一章列出的直接依赖；缺包或版本不符时，先保留完整错误输出。

CHAPTER 04

日常命令

npm 脚本只是项目预先写好的短命令。先看它展开成什么，再判断它能证明什么。

从仓库根目录运行	实际脚本	结果
<code>npm --prefix frontend run dev</code>	<code>vite</code>	启动本机 Vite 页面；进程持续运行， Control + C 停止。
<code>npm --prefix frontend run test</code>	<code>vitest run</code>	单次运行全部前端测试，结束后退出。
<code>npm --prefix frontend run test -- App.test.js</code>	<code>vitest run App.test.js</code>	把 <code>--</code> 后面的文件名交给 Vitest，只跑相关测试。
<code>npm --prefix frontend run build</code>	<code>vite build</code>	检查生产构建并写入 <code>frontend/dist/</code> 。

拆开命令语法

```
npm  --prefix frontend  run  test  --  App.test.js
|      |                |    |    |
工具  npm 项目目录    跑脚本 脚本名 分隔符 交给 Vitest
```

`npm run` 会把本地 `node_modules/.bin` 加入 PATH，所以本项目不需要 全局安装 Vite、Vitest 或 Tauri CLI。脚本清单的唯一事实源是 `frontend/package.json`；仓库没有定义 `npm start`。

DEV 不是完整应用

`npm run dev` 只监听 `127.0.0.1:1420`。端口已被占用时，`strictPort` 会让它失败，而不是偷偷换端口；先找出占用者，不要随手改配置。

走到完整桌面应用

Tauri 开发命令会启动 Vite 和 Rust 宿主，但不会替你制造 Python Sidecar；缺一层就不是完整链路。

最短开发启动路径

01 安装前端依赖

```
npm --prefix frontend ci
```

02 构建 Python Sidecar

```
.venv/bin/python scripts/build_sidecar.py
```

03 启动桌面应用

```
npm --prefix frontend run tauri:dev
```

第三步展开为 `tauri dev`。Tauri 读取 `frontend/src-tauri/tauri.conf.json`，先执行 `npm run dev`，再让 Rust 宿主连接 `http://127.0.0.1:1420`，并启动已存在的 Sidecar 二进制。

为什么 Sidecar 必须先构建

Tauri 配置声明外部二进制 `binaries/keikeu-sidecar`；真正生成的文件名会带当前 Rust target triple。npm 既不安装 PyInstaller，也不运行 `scripts/build_sidecar.py`，所以缺 Sidecar 时应回到第二步，而不是重装 Vue 依赖。

生产构建入口

```
.venv/bin/python scripts/build_sidecar.py  
npm --prefix frontend run tauri:build
```

`tauri:build` 会自动执行 Vite build，再调用 Cargo/Tauri 组装桌面产物。它比 `npm run build` 多，但仍不自动证明签名、公证、真实 Vault smoke 或产品验收通过。

保护作者数据

启动真实桌面应用可能读取已选择的 Vault 与本地应用状态。测试迁移、删除、恢复或持久配置时，先用 fixture、合成 Vault 或副本；npm 命令不会替你隔离真实作者内容。

依赖变更纪律

新依赖不是修复手段，而是新的维护责任。先证明现有代码、平台能力和已安装包都不够，再增加。

默认：不要改依赖树

- 不为“以后可能会用”安装包；
- 不全局安装项目已经本地声明的 CLI；
- 不运行 `npm update` 做顺手升级；
- 不运行 `npm audit fix` 自动改树；
- 不删除 lockfile 后重新解析“碰碰运气”。

本项目直接依赖全部使用精确版本。版本范围前的 `^` 或 `~` 会改变升级语义，不应在无设计决定时顺手加入。

任务明确批准增加依赖时

```
# 运行时依赖
npm --prefix frontend install --save-exact <package>@<exact-version>

# 只用于构建或测试
npm --prefix frontend install --save-dev --save-exact <package>@<exact-version>
```

1. 先说明为什么现有代码、浏览器/Tauri 原生能力和当前依赖不够。
2. 确认它属于 `dependencies` 还是 `devDependencies`。
3. 只审阅 `frontend/package.json` 与 lockfile 的预期差异。
4. 重新运行前端测试与构建；跨桌面边界时再跑相应 Cargo/Tauri 检查。

安全报告不是自动改动授权

`npm --prefix frontend audit --omit=dev` 可以查询生产依赖报告，但它访问 registry，结果只代表查询时刻。发现告警后先确认受影响路径和升级范围；不要把 `npm audit fix` 当成无害清理。

故障与证据边界

先按错误属于哪一层分类。重装所有东西通常只会抹掉线索，并不能修复根因。

最短故障表

症状	最可能原因	下一步
找不到 <code>package.json</code>	在仓库根目录漏了 <code>--prefix frontend</code>	回到根目录，使用本手册中的完整命令。
<code>EBADENGINE</code> 或版本警告	实际 Node/npm 不符合 manifest；也可能被旧 <code>.node-version</code> 带到 22.23.1	先记录 <code>node --version</code> 与 <code>npm --version</code> ，校准到 22.23.2 / 10.9.8。
<code>npm ci</code> 说 manifest 与 lock 不一致	声明文件真的发生了漂移	查看 Git diff；不要让 <code>npm install</code> 静默重写现场。
1420 端口占用	旧 Vite/Tauri 进程仍在运行	确认并停止占用者；不要绕开 <code>strictPort</code> 。
浏览器页面能开，桌面调用失败	只运行了 Vite，没有 Tauri bridge	先构建 Sidecar，再运行 <code>tauri:dev</code> 。
Tauri 报 Sidecar 不存在	Python 二进制尚未为当前 target 构建	运行 <code>.venv/bin/python scripts/build_sidecar.py</code> ，保留原错误。

每条命令到底证明什么

检查	可以声称	不能顺带声称
<code>npm run test</code>	当前 Vitest 测试通过。	Python、Rust、真机或产品验收通过。
<code>npm run build</code>	当前 Vue/Vite 生产构建完成。	桌面包可以启动。
<code>npm run tauri:dev</code>	在实际观察到的路径上，开发宿主能够启动。	签名、公证、安装包或真实作者场景通过。
<code>npm run tauri:build</code>	当前本机桌面构建完成。	发布、兼容性或产品接受已经成立。

新人完成一次前端改动的最短检查

```
npm --prefix frontend run test
npm --prefix frontend run build
git diff --check
```

UI 行为变化还需要聚焦的 Vitest 与适当的 Tauri smoke；文档检查不能替代应用检查。未运行的检查要明说。

SOURCES

事实来源

1. `frontend/package.json`：版本、直接依赖与脚本。
2. `frontend/package-lock.json`：lockfile v3 与完整依赖树。
3. `frontend/vite.config.js`：端口、strictPort 与 jsdom。
4. `frontend/src-tauri/tauri.conf.json`：Vite 前置命令与 Sidecar 声明。
5. `scripts/build_sidecar.py`：Sidecar 构建与 target 命名。
6. CP13 macOS workflow：冻结安装、测试与桌面构建顺序。
7. `.gitignore`：前端与桌面生成物边界。
8. README、PROJECT 与 Road v0.6 计划书：当前工具链契约与检查命令。
9. `.node-version`：仍为 22.23.1 的已知漂移。
10. Node.js 22.23.2 官方发布页。
11. npm CLI v10: `npm ci`。
12. npm CLI v10: `npm run-script`。
13. npm CLI v10: `npm audit`。
14. npm CLI v10: `package.json`。
15. npm CLI v10: `package-lock.json`。

Truth boundary

本页是非规范性新人手册，不创造版本、依赖、Git、发布或真实 Vault 操作权限。当前状态以 PROJECT 为准，工程与证据规则以 RULES 为准，实际脚本与依赖以当前源码为准。若它们与本页冲突，本页错；修本页，不复制一套新规则。